**COURSE:**

**EC-360 SOFTWARE ENGINEERING**

**SUBMITTED TO:**

**DR FAROOQUE AZAM**

**SUBMITTED BY:**

**Group Members:**

| | | |
|---|---|---|
| **1.** | **HANIA SYED** | **405339** |
| **2.** | **SHAFLA REHMAN** | **412780** |
| **3.** | **MUHAMMAD JUNAID** | **432003** |
| **4.** | **FAIQA UMMARAH AHMAD** | **405052** |
| **Degree** | **44** | |
| **Syndicate** | **A** | |

**Date of Submission:** **12TH-April -2026**

**TABLE OF CONTENTS**

## Contents

1: Introduction ..... 4
1.1 Purpose of Survey ..... 4
1.2 CLO-3 and PLO-12 Alignment ..... 4
2: Survey Methodology ..... 5
2.1 Organizations and Respondents ..... 5
3: Findings & Analysis ..... 5
3.1 Development Methodologies ..... 5
3.1.1 SDLC Models in Use ..... 5
3.1.2 Factors Influencing Methodology Choice ..... 6
3.1.3 Hybrid and Agile Adoption ..... 6
3.1.4 Safety vs. Enterprise ..... 6
3.2 Architecture and Design Practices ..... 6
3.2.1 Architecture Design Approach ..... 6
3.2.2 Trending Architectural Styles ..... 6
3.2.3 Design Pattern Usage ..... 7
3.3 Coding Standards and Quality Practices ..... 7
3.3.1 Coding Standards Enforcement ..... 7
3.3.2 Code Review Practices ..... 7
3.3.3 Quality and Maintainability Approaches ..... 7
3.4 Testing and Quality Assurance ..... 8
3.4.1 Testing Types in Practice ..... 8
3.4.2 CI/CD and QA Tools ..... 8
3.4.3 Automated Regression Testing ..... 8
3.5 Maintenance and Sustainability ..... 9
3.5.1 Common Maintenance Challenges ..... 9
3.5.2 Future-Proofing Strategies ..... 9
4. Discussion of Trends ..... 9
4.1 Emerging Technologies and Practices ..... 9
4.1.1 AI-Assisted Development ..... 9

4.1.2 DevOps Culture ..... 10
4.1.3 AI-Based Testing ..... 10
4.2 Comparison with Academic Curriculum ..... 10
4.2.1 Areas of Alignment ..... 10
4.2.2 Gaps and Opportunities ..... 11
4.3 Lifelong Learning Implications (CLO-3 → PLO-12) ..... 11
4.3.1 Rapid Technological Evolution ..... 11
4.3.2 Tool and Platform Shifts ..... 11
4.3.3 Methodological Adaptation ..... 11
4.3.4 Professional Development ..... 11
5. Conclusion ..... 12
5.1 Key Insights ..... 12
5.2 Student Reflection on Lifelong Learning ..... 12
6. References ..... 12
7. Appendices ..... 13

## Group #10: Reporting New Trends in Software Development Methodologies


### *Executive Summary:*

This analytical report presents findings from a comprehensive survey of five software development professionals across various organizations including **Nasdaq Inc., NSTP, Ismail Industries, Mars Digital, and DRM**. The survey was conducted between April 9-11, 2026, to identify and analyze emerging trends in software development methodologies, aligning with CLO-3 (Course Learning Outcome 3) and PLO-12 (Program Learning Outcome 12) focused on lifelong learning and technological adaptation.

Key findings reveal a strong industry shift toward Agile and hybrid methodologies, microservices architecture, comprehensive testing practices including AI-based testing, and emerging technologies such as DevOps, AI-assisted development tools (like GitHub Copilot), and cloud-native solutions. The survey demonstrates that modern software development prioritizes flexibility, automation, quality assurance, and continuous improvement; critical insights for students pursuing careers in software engineering.


# 1: Introduction

## 1.1 Purpose of Survey

The purpose of this survey is to identify and analyze emerging trends in software development methodologies used in the industry. It supports CLO-3 by helping students report new trends and contributes to PLO-12 by promoting lifelong learning.
By interacting with industry professionals, the survey bridges the gap between academic knowledge and real-world practices.

## 1.2 CLO-3 and PLO-12 Alignment

CLO-3 (reporting new trends in software development) is aligned with PLO-12 (lifelong learning). This ensures that students not only understand current industry practices but also develop skills to continuously learn and adapt to new technologies.
The survey highlights the importance of staying updated and evolving in the field of software engineering.

# 2: Survey Methodology

## 2.1 Organizations and Respondents

The survey was conducted with five software development professionals from diverse organizational contexts:

| Organization | Role | Experience | Key Tools |
|---|---|---|---|
| Nasdaq Inc. | QA / Tester | 5-10 years | Jira, Jenkins, GitLab CI/CD |
| NSTP | Software Developer | 0-2 years | GitLab, Docker, Playwright |
| Ismail Industries | Software Developer | 0-2 years | Jira, GitHub Actions |
| Mars Digital | Software Developer | 0-2 years | GitLab, Jenkins |
| DRM | Software Developer | 0-2 years | Azure, Docker, Kubernetes |

**Mode:** Virtual Connection & Google Form Distribution

**Tools Used for Analysis:** Google Sheets (for raw data parsing), Qualitative Pattern Analysis.

# 3: Findings & Analysis

## 3.1 Development Methodologies

### 3.1.1 SDLC Models in Use

The survey reveals a clear industry preference for Agile methodologies, with 60% of respondents utilizing Agile (Scrum) as their primary development model. This finding aligns with contemporary industry trends emphasizing iterative development, flexibility, and rapid delivery cycles.

| SDLC Model | Count | Organizations |
|---|---|---|
| Agile (Scrum) | 3 (60%) | Nasdaq Inc., NSTP, Ismail Industries |
| Waterfall | 2 (40%) | NSTP, DRM |
| DevOps | 2 (40%) | Mars Digital, DRM |
| Lean Development | 1 (20%) | Ismail Industries |

### 3.1.2 Factors Influencing Methodology Choice

The choice of development methodology mainly depends on project needs. The most important factor is **project complexity (80%)**, followed by **client requirements (60%)** and **budget constraints (40%)**. **Time constraints (20%)** also play a role, especially in fast-paced projects. Overall, organizations select methodologies based on project scope and business demands.

### 3.1.3 Hybrid and Agile Adoption

Many organizations use a mix of Agile and traditional methods, especially for large or critical projects. About **40% use hybrid approaches**, while **40% rely on pure Agile**. For safety-critical systems, stricter processes are followed, whereas more flexible Agile methods are used for regular enterprise applications. This shows that methodology choice depends on project type.

### 3.1.4 Safety vs. Enterprise

Nasdaq Inc. (QA/Tester) highlighted a "Strict process for safety-critical systems," whereas enterprise systems (like at Ismail Industries) allow for "Flexible Agile." This confirms that risk dictates rigor.

## 3.2 Architecture and Design Practices

### 3.2.1 Architecture Design Approach

All respondents who provided information (100%) indicated that architecture is formally designed before implementation, demonstrating a universal industry practice of upfront architectural planning. This finding contradicts the misconception that Agile methodologies eliminate design phases; instead, it shows that modern development practices integrate architectural planning within iterative frameworks.

### 3.2.2 Trending Architectural Styles

Survey results reveal a significant shift toward modern architectural patterns:

| Architecture Style | Adoption | Analysis |
|---|---|---|
| Microservices Architecture | 40% | Highest trending style; scalability focus |
| Monolithic (Modular) | 40% | Still prevalent; simplified deployment |
| Serverless Architecture | 20% | Emerging; cloud-native approach |

The microservices architecture adoption (40%) represents a significant industry shift toward distributed, independently deployable services. This trend enables organizations to achieve greater scalability, fault isolation, and technology diversity. Notably, Nasdaq Inc. and DRM explicitly identified microservices as their architectural approach, while traditional monolithic architectures remain common (40%) in organizations like Ismail Industries and Mars Digital.

### 3.2.3 Design Pattern Usage

Design patterns are moderately adopted across the surveyed organizations. MVC (Model-View-Controller) emerged as the most common pattern (40%), followed by Dependency Injection (20%). Notably, some organizations (40%) reported rarely using formal design patterns, suggesting a gap between academic emphasis on design patterns and practical application in certain industry contexts.

## 3.3 Coding Standards and Quality Practices

### 3.3.1 Coding Standards Enforcement

Coding standards enforcement varies significantly across organizations:

- **Organization-wide standards:** 20% (DRM)
- **Project-level standards:** 40% (Ismail Industries, Mars Digital)
- **No strict enforcement:** 20% (Nasdaq Inc.)
- **Not specified:** 20% (NSTP)

This variation indicates that while coding standards are recognized as important, their enforcement level depends on organizational maturity, project requirements, and team size. Larger organizations and critical projects tend toward stricter standardization.

### 3.3.2 Code Review Practices

Code review is universally practiced among respondents who provided information, with varying approaches:

| Review Method | Adoption | Organizations |
|---|---|---|
| Peer Review | 40% | Nasdaq Inc., Ismail Industries |
| Pull Request Reviews | 40% | Mars Digital, DRM |
| Automated Code Analysis | 20% | DRM |

### 3.3.3 Quality and Maintainability Approaches

Organizations employ various strategies to ensure code quality and maintainability:

- **Code Reviews:** 60% - Primary quality assurance mechanism
- **Clean Code Principles:** 40% - Emphasis on readability and simplicity
- **Documentation:** 40% - Critical for maintainability
- **Refactoring:** 20% - Continuous improvement approach

The emphasis on Clean Code principles (popularized by Robert C. Martin) indicates a shift toward writing self-documenting, maintainable code. The combination of code reviews and clean coding practices creates a robust quality assurance framework.

## 3.4 Testing and Quality Assurance

### 3.4.1 Testing Types in Practice

The survey reveals comprehensive testing practices across organizations, with multiple testing levels employed simultaneously:

| Testing Type | Usage | Significance |
|---|---|---|
| Unit Testing | 60% | Foundation of test pyramid |
| Integration Testing | 60% | Component interaction verification |
| UAT (User Acceptance Testing) | 60% | Client validation critical |
| Automated Testing | 40% | Efficiency and regression prevention |
| System Testing | 40% | End-to-end validation |
| Performance Testing | 20% | Scalability assurance |

The high adoption of unit testing (60%), integration testing (60%), and UAT (60%) demonstrates adherence to the testing pyramid concept, where comprehensive testing at multiple levels ensures software quality. Nasdaq Inc., with 5-10 years of mature QA practices, employs the most comprehensive testing strategy including all six testing types.

### 3.4.2 CI/CD and QA Tools

Organizations widely use CI/CD and QA tools:

- GitHub Actions: 60%
- Jenkins: 40%
- GitLab CI/CD: 20%
- Azure Pipelines: 20%
- Playwright: 20%
- Selenium: 20%

GitHub Actions is the most commonly used tool, showing a shift toward modern, integrated workflows. At the same time, tools like Jenkins are still in use, indicating a mix of old and new technologies.

### 3.4.3 Automated Regression Testing

Automated regression testing adoption varies significantly: 40% report widespread use (Nasdaq Inc., Mars Digital), 20% use it rarely (Ismail Industries), and 20% employ it sometimes (NSTP). The correlation between organizational maturity and automated testing adoption is evident, with the most experienced respondent (Nasdaq Inc., 5-10 years) reporting the highest automation levels.

## 3.5 Maintenance and Sustainability

### 3.5.1 Common Maintenance Challenges

Survey respondents identified several critical challenges in software maintenance:

| Challenge | Frequency | Impact |
|---|---|---|
| Changing Requirements | 40% | Requires architecture flexibility |
| Scalability Issues | 40% | Drives microservices adoption |
| Technical Debt | 20% | Accumulates without refactoring |

Changing requirements (40%) and scalability issues (40%) emerged as the dominant maintenance challenges. These findings reinforce the need for flexible architectures (like microservices) and adaptive methodologies (like Agile) that can accommodate evolving business needs.

### 3.5.2 Future-Proofing Strategies

Organizations employ various strategies to ensure long-term software sustainability:

- **Cloud Adoption:** 40% - Scalability and flexibility through cloud infrastructure
- **Modular Architecture:** 20% - Independent component evolution
- **CI/CD Pipelines:** 20% - Automated, reliable deployments
- **Documentation:** 20% - Knowledge preservation and transfer

Cloud adoption emerged as the leading future-proofing strategy (40%), reflecting the industry-wide shift toward cloud-native architectures. This trend enables organizations to scale resources dynamically, reduce infrastructure costs, and leverage managed services for improved reliability and performance.

# 4. Discussion of Trends

## 4.1 Emerging Technologies and Practices

### 4.1.1 AI-Assisted Development

One of the most significant emerging trends identified in the survey is AI-assisted development, with 80% of respondents citing AI-based tools as a future trend. Specifically, GitHub Copilot and similar AI pair programming tools are being adopted to enhance developer productivity.

| Emerging Technology | Adoption | Impact |
|---|---|---|
| AI-Assisted Development (Copilot) | 80% | Enhanced productivity, code generation |
| AI-Based Testing | 60% | Intelligent test case generation, defect prediction |

| | | |
|---|---|---|
| DevOps | 60% | Continuous integration and deployment culture |
| Microservices | 20% | Distributed, scalable architecture |
| Low-Code/No-Code Platforms | 20% | Democratizing development, rapid prototyping |
| MLOps | 20% | Machine learning lifecycle management |

The widespread recognition of AI-assisted development (80%) signals a fundamental shift in how software is created. AI-powered coding assistants like GitHub Copilot can generate code snippets, suggest completions, identify bugs, and accelerate development workflows. This trend represents a paradigm shift where developers increasingly collaborate with AI systems to enhance productivity.

#### 4.1.2 DevOps Culture

DevOps practices were cited by 60% of respondents as a dominant future trend, indicating the industry's move toward breaking down silos between development and operations teams. The DevOps culture emphasizes:

- Continuous Integration and Continuous Deployment (CI/CD)
- Infrastructure as Code (IaC)
- Automated testing and deployment pipelines
- Collaborative culture between teams
- Monitoring and observability

The adoption of DevOps aligns with the broader industry trend toward faster release cycles, improved software quality, and enhanced collaboration. Organizations practicing DevOps can deploy code multiple times per day, significantly accelerating innovation and responsiveness to market changes.

#### 4.1.3 AI-Based Testing

AI-based testing, cited by 60% of respondents, represents the application of machine learning and artificial intelligence to quality assurance. These technologies can automatically generate test cases, predict defect-prone areas, optimize test suites, and provide intelligent test prioritization. This trend addresses the challenge of maintaining comprehensive test coverage as software systems grow increasingly complex.

## 4.2 Comparison with Academic Curriculum

The survey shows both alignment and gaps between academic learning and industry practices.

#### 4.2.1 Areas of Alignment

- SDLC models like Agile and Waterfall are widely used in industry

- Testing practices (unit, integration, system) match academic concepts
- Design patterns (e.g., MVC) are applied in real projects
- Clean code and refactoring principles are followed

#### 4.2.2 Gaps and Opportunities

- Limited exposure to AI-assisted development and testing
- Lack of hands-on experience with DevOps (CI/CD pipelines)
- Minimal focus on cloud-native systems (microservices, serverless, containers)
- Limited use of modern tools (GitHub Actions, Docker, Kubernetes, Jira)

These gaps suggest that students need more practical exposure to modern tools, cloud platforms, and automation technologies.

### 4.3 Lifelong Learning Implications (CLO-3 → PLO-12)

The survey highlights the importance of continuous learning in software engineering.

#### 4.3.1 Rapid Technological Evolution

Technologies like AI tools, serverless systems, and MLOps are rapidly emerging, showing that developers must constantly adapt.

#### 4.3.2 Tool and Platform Shifts

The shift from tools like Jenkins to GitHub Actions shows that engineers must regularly learn new tools and platforms.

#### 4.3.3 Methodological Adaptation

Organizations use different methodologies depending on the project. Developers must understand and adapt multiple approaches.

#### 4.3.4 Professional Development

Students can improve by:

- Following industry blogs and research
- Participating in communities and open-source projects
- Experimenting with new technologies
- Attending workshops and training

# 5. Conclusion

### 5.1 Key Insights

This survey of five software development professionals across diverse organizations reveals several critical trends shaping contemporary software engineering:

- **Agile Dominance:** Agile methodologies (60% adoption) and hybrid approaches dominate modern software development, reflecting the industry's emphasis on flexibility and iterative delivery.
- **Microservices Shift:** 40% of organizations are adopting microservices architecture, driven by scalability requirements and the need for independent service deployment.
- **Comprehensive Testing:** Organizations employ multi-layered testing strategies (unit, integration, UAT, automated) with strong CI/CD pipeline integration.
- **AI Integration:** 80% of respondents identify AI-assisted development and 60% cite AI-based testing as dominant future trends, signaling a fundamental shift in software creation.
- **DevOps Culture:** 60% adoption of DevOps practices demonstrates the industry's commitment to continuous integration, deployment automation, and collaborative development.
- **Cloud-Native Focus:** Cloud adoption (40% as future-proofing strategy) and serverless architecture (20%) reflect the industry's migration toward scalable, managed infrastructure.

### 5.2 Student Reflection on Lifelong Learning

This survey emphasizes the importance of continuous learning in software engineering. While academic courses provide a strong foundation, many modern tools and practices such as cloud computing, CI/CD, and AI tools require self-learning and practical experience. It also shows that technologies and methodologies evolve rapidly, making it essential for students to stay updated. As future software engineers, we must actively explore new tools, work on projects, and engage with industry trends to remain relevant and grow professionally.

# 6. References


*[1] Pressman, R. S., & Maxim, B. R. (2020).* Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

*[2] Sommerville, I. (2021).* Software Engineering (10th ed.). Pearson.

*[3] Martin, R. C. (2008).* Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

*[4] Kim, G., Humble, J., Debois, P., & Willis, J. (2016).* The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations. IT Revolution Press.

*[5] Newman, S. (2021).* Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.

*[6] Forsgren, N., Humble, J., & Kim, G. (2018).* Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.

[7] GitHub Blog. (2023). "AI-Powered Developer Tools: The Future of Coding." Retrieved from https://github.blog

[8] Atlassian. (2024). "Agile Best Practices." Retrieved from https://www.atlassian.com/agile

[9] AWS. (2024). "Serverless Architecture." Retrieved from https://aws.amazon.com/serverless

[10] ISTQB. (2023). "Software Testing Trends and Emerging Technologies." Retrieved from https://www.istqb.org

# 7. Appendices

## Appendix A: Raw Survey Responses

**Note:** The raw survey data in CSV format (collected using google forms) is attached separately as "Survey_Data(Grp#10).csv" for detailed examination and further analysis.